Statystyka - laboratorium 1

Poniższy scenariusz zajęć powstał w oparciu o materiały Michała Ciacha i Łukasza Rajkowskiego.

Na dzisiejszych zajęciach zapoznamy się z pakietem **R** i ze środowiskiem **RStudio**.

Co to R i po co to?

R jest jednym z najpopularniejszych programów do analizy danych (obok SAS, Microsoft Excel i języka Python). Jego główną zaletą jest to, że jest mocno przystosowany do szybkiego przeprowadzania prostych analiz z użyciem gotowych pakietów.

Przystosowanie do konkretnego zadania oznacza jednocześnie, że **R** idzie na przekór większości konwencji spotykanych w innych językach programowania. Z tego powodu, o ile **R** jest bardzo wygodnym narzędziem do przeprowadzania rutynowych analiz, to implementowanie własnych algorytmów w **R** jest bardzo uciążliwe i mało wydajne obliczeniowo.

Głównym konkurentem **R** jest język Python. Oba narzędzia oferują komplementarne możliwości. O ile **R** jest wyspecjalizowanym pakietem statystycznym, to język Python jest językiem ogólnego zastosowania. Z tego powodu dużo lepiej nadaje się do implementowania własnych, niestandardowych rozwiązań. Jest również dużo bardziej wydajny obliczeniowo - kod napisany w tym języku wywołuje się kilku(dziesięcio)krotnie szybciej niż analogiczny kod napisany w **R**. Z drugiej strony, przeprowadzenie typowej analizy i przygotowanie raportu statystycznego zajmuje w nim kilku(dziesięcio)krotnie więcej czasu.

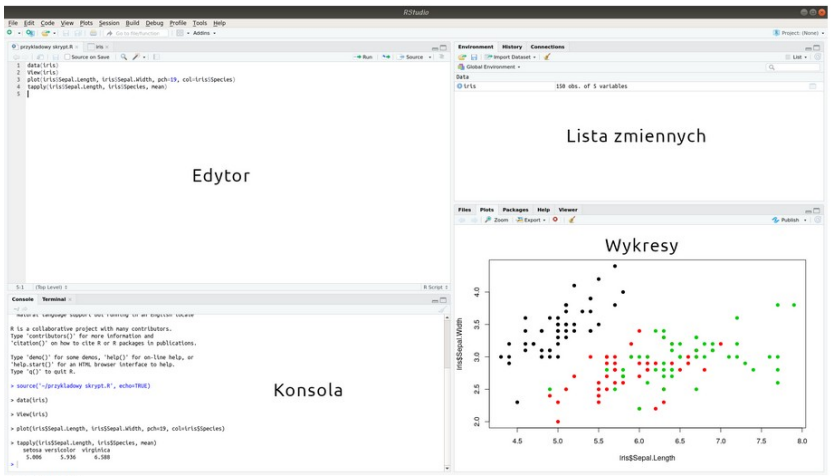
Zarówno **R** jak i Python są powszechnie używane w analizie danych i każdy, kto planuje w przyszłości pracować z danymi, powinien poznać oba te narzędzia przynajmniej w podstawowym stopniu i wiedzieć, kiedy stosować każde z nich.

RStudio

Uruchom program **RStudio**.

RStudio jest zintegrowanym środowiskiem programistycznym (*integrated development environment, IDE*) służącym do przeprowadzania analiz statystycznych

i tworzenia grafiki naukowej przy pomocy pakietu **R**.



Okno Rstudio

Środowisko **RStudio** składa się z następujących paneli:

- *Edytor*, służący do pisania skryptów,
- *Konsola*, służąca do wywoływania pojedyńczych poleceń,
- *Lista zmiennych* utworzonych przez użytkownika,
- panel z *Wykresami*.

W zakładkach znajdują się dodatkowe okna, m.in. historia wywołanych poleceń i dokumentacja funkcji i pakietów.

R jako kalkulator

R to taki duży kalulator. Aby się o tym przekonać utwórz najpierw nowy skrypt (*File* → *New File* → *R Script*) i zapisz go w wybranym katalogu. Następnie wpisz do niego wymienione poniżej komendy (każdą w osobnej linijce), wywołaj je po kolei i przyjrzyj się wynikom. Pojedynczą linijkę skryptu możesz wywołać w konsoli wciskając `Ctrl+Enter`, a cały skrypt klikajac przycisk `Source` w prawym górnym rogu okna edytora.

- `2+2`
- `sin(.3)`
- `log(2)`
- `?log`, wyświetla pomoc dla funkcji `log`

- `pi`
- `exp(1)`

Zadanie. Jaka jest podstawa logarytmu? Jak ją zmienić?

Oczywiście można dokonywać przypisywania wartości do zmiennych. Na przykład `x = sqrt(19)` przypisze zmiennej `x` wartość pierwiastka z 19. Natomiast `y <- sin(10)` przypisze zmiennej `y` wartość sin(10). Możemy potem dodać te dwie wartości do siebe `x+y`.

Operatory przypisania `=` oraz `<-` są w powyższym kontekście równoważne. Dla zwiększenia czytelności zaleca się stosowanie `<-` zamiast `=` do przypisywania wartości do zmiennej.

Wektory i operacje na nich

R umożliwia wygodne operowanie na wektorach. Nowy wektor wypełniony kolejno liczbami $1, 4, 2, 6, 7, 8.2$ tworzymy komendą `x <- c(1,4,2,6,7,8.2)`.

Można również łatwo wygenerować wektory kolejnych liczb całkowitych, i ogólniej, ciągi arytmetyczne. W szczególności bardziej złożone sekwencje liczb możemy tworzyć komendą `seq()`. Jej działanie możesz sprawdzić wpisując w konsolę komendę `help(seq)`. Wyświetli ona dokumentację funkcji `seq` w miejscu panelu z wykresami. Na przykład:

- wektor złożony z kolejnych liczb całkowitych od 3 do 10 tworzymy komendą `3:10`
- komenda `seq(5,17, by=.3)` tworzy ciąg arytmetyczny od 5 do 17 z różnicą .3
- komenda `seq(5,17, length.out=100)` tworzy ciąg arytmetyczny długości 100 od 5 do 17

Większość funkcji arytmetycznych w **R** jest *zwektoryzowana*, to znaczy możliwa do zastosowania na wektorach liczb. Jako wynik dostaniemy wektor wyników na współrzędnych. Przetestuj komendy: `x^2` oraz `sin(x)`. Porównania też są *zwektoryzowane*, na przykład jako wynik komendy `x > 3` dostaniemy wektor odpowiedzi na pytanie, czy dana współrzędna jest większ od 3.

Wbudowane są oczywiście podstawowe statystyki dotyczące wektorów. Wpisz poniższe komendy (każdą w osobnej linijce), wywołaj je po kolei i przyjrzyj się wynikom.

- `mean(x)`
- `sum(x)`
- `sum(x>3)` powie ile współrzędnych jest większych od 3
- `cumsum(x)`

- `min(x)`
- `max(x)`
- `median(x)`
- `quantile(x, probs=.2)`
- `range(x)`
- `var(x)`
- `sd(x)` - odchylenie standardowe

A co jeśli chcemy się dostać do konkretnych współrzędnych wektora? Służą do tego nawiasy kwadratowe. Wpisz poniższe komendy (każdą w osobnej linijce), wywołaj je po kolei i przyjrzyj się wynikom.

- `x[3]`
- `x[3:5]` - zwróci wyrazy od trzeciego do piątego
- `x[3:5] <- pi^(1:3)` - podmieni wyrazy
- `x[-2]` - zwróci wektor `x` bez drugiego wyrazu

Bardzo przydatne jest to, że do nawiasów kwadratowych możemy wrzucić wektor logiczny o długości takiej, jak `x`. Przetestuj komendę `x[x>3]]`.

1 R jako generator liczb losowych

R ma wbudowane funkcje pozwalające na losowanie z popularnych rozkładów prawdopodobieństwa. Na przykład

- `runif(1)` zwróci liczbę wylosowaną jednostajnie z przedziału $[0, 1]$
- `runif(10)` zwróci 10 liczb wylosowanych niezależnie, jednostajnie z $[0, 1]$
- `runif(10, min=3, max=7)` zwróci 10 liczb wylosowanych niezależnie, jednostajnie z $[3, 7]$
- `rnorm(5, mean=3)`

Dla tych rozkładów prawdopodobieństwa najczęściej mamy też funkcje zwracające ich gęstości/dystrybuanty/kwantyle (odpowiednio z przedrostkami `d`/`p`/`q`). Przetestuj następujące komendy: `pnorm(2)`, `dnorm(3)`, `qnorm(.2)`.

Wyjątkiem od zasady nazewniczej `rnazwa` jest losowanie współrzędnej wektora. Na przykład:

- `sample(x,1)` wylosuje jedną współrzędną `x`
- `sample(x,3)` wylosuje (bez zwracania) trzy współrzędne `x`

- `sample(x,100,replace=T)` wylosuje sto współrzędnych `x` (ze zwracaniem)
- `sample(x, length(x))` zwróci permutację wyrazów `x`
- `sample(1:6, 100, replace=T)` wynik 100 rzutów kostką sześcienną

2 Podstawowa analiza danych

Strukturą danych, która w **R** odpowiada za reprezentację klasycznych, *tabelkowych* danych jest ramka danych (`data.frame`). Przyjrzyjmy się wbudowanej ramce danych `mtcars`. Wpisz poniższe komendy (każdą w osobnej linijce), wywołaj je po kolei i przyjrzyj się wynikom.

- `mtcars`
- `?mtcars` - `mtcars` jako wbudowana ramka ma swoją dokumentację
- `dim(mtcars)` - wymiary ramki
- `ncol(mtcars)` - l. kolumn
- `nrow(mtocars)` - l. wierszy
- `colnames(mtcars)` - nazwy kolumn
- `head(mtcars)` - "podgląd", tzn. pierwszych sześć wierszy

Dostęp do elementów ramki i jej „podramek" odbywa się analogicznie do dostępu wyrazów wektora. Na przykład:

- `mtcars[ 2, 5 ]` - wyraz w drugim wierszu i piątej kolumnie
- `mtcars[ c(1,3), ]` - wiersze 1 i 3
- `mtcars[ , c(2,4) ]` - kolumny 2 i 4
- `mtcars[ c(1,3), c(2,4) ]`
- `mtcars[ ,c("mpg", "carb") ]` - kolumny `mpg` i `carb`
- `mtcars["carb"]` - kolumna `carb`
- `mtcars$carb` - też kolumna `carb`
- `mtcars[ mtcars$mpg>2, ]`

Zadanie. Wyświetl informacje o samochodach, które maja więcej niż 4 cylindry.

Funkcja `summary` wyświetla podstawowe informacje o kolumnach. Można wówczas zauważyć, że kolumny `vs` i `am` potraktowane są jako kolumny numeryczne (zer i jedynek). To jednak nie ma specjalnego sensu - te zera i jedynki służą do kodowania pewnej informacji (np. dla `vs` jest to informacja, czy silnik jest V-kształtny, czy prosty). Takie zmienne dużo lepiej przekonwertować od razu na typ `factor`. Przetestuj kolejno komendy:

- `summary(mtcars)`
- `mtcars$vs <- as.factor(mtcars$vs)`
- `summary(mtcars)` teraz dla `vs` mamy tylko liczbę zer i jedynek

Oczywiście możemy wczytywać dane do **R** z różnych innych formatów. Na przykład komenda `read.csv("ścieżka/do/pliku.csv")` wczytuje plik csv.

Zadania

1. Sprawdź czy $e^{\pi\sqrt{43}}$ jest liczbą całkowitą.
2. Wyznacz eksperymentalnie wartość π. W tym celu wylosuj N punktów z kwadratu $[-1,1]^2$ i zobacz, jaka część trafia we wpisane w ten kwadrat koło. Jak jakość przybliżenia zależy od N?
3. Z ciągu $9, 15, 2, 18, 16, 4, 19, 17, 6, 13, 3, 12, 7, 5, 10$ wybierz liczbę: najmniejszą, średnią oraz medianę.
4. (zadanie zaczerpnięte ze skrypyu Mikołaja Rybińskiego) Odpowiedz na poniższe pytania dotyczące zbioru `mtcars`:
   (a) Ile wynosi maksymalny przebieg (w milach/galon) i który samochód go osiągnął?
   (b) Jak wygląda pierwsza trójka samochodów o największej liczbie koni mechanicznych?
   (c) Jakie są średnie przyspieszenia dla:
      - wszystkich samochodów,
      - samochodów z/bez automatycznej skrzyni biegów,
      - mercedesów,
      - czołowych 20% samochodów po względem liczby koni mechanicznych.
5. Plik `h3units.csv` zawiera informacje o jednostkach spotykanych w grze Heroes 3. Na podstawie tych informacji odpowiedz na poniższe pytania:
   (a) Jakie są możliwe nazwy zamków (kolumna `Castle`) w grze?
   (b) Jaka jest mediana wartości ataku dla jednostek z **Nekropolis**?
   (c) Jaka jest średnia wartość obrony dla zaawansowanych (tzn. zawierających "+" w kolumnie `Level`) jednostek dla Twierdzy (**Stronghold**)?
   (d) Wyznacz średnią szybkość jednostek, które nie latają, w podziale na zamki.